



Integração CAv4 + Microsoft Entra ID

Guia técnico de reimplimentação

Índice

1. Visão simples do fluxo	3
3. Variáveis de ambiente	4
2. Arquitetura e arquivos	5
4. Endereços e callbacks	6
5. Passo a passo completo do login	7
6. Endpoints do CAV4	8
6.1 Permissões e autorização do CAV4	9
7. Endpoints do Entra ID e Microsoft Graph	10
8. Permissões do Entra ID e Microsoft Graph	11
8.1 Orquestração e formato final	12
9. Front-end e rotas da própria POC	13
11. Checklist para outro projeto	14

gens# Guia de reimplementação: CAV4/CA + Microsoft Entra ID

Este guia foi escrito a partir do código real da POC, principalmente `backend/auth.py`, `backend/oidc.py`, `backend/ca_client.py`, `backend/graph_client.py`, `backend/config.py`, `backend/session.py`, `frontend/app/page.tsx` e `frontend/next.config.ts`.

Nota de nomenclatura: neste documento, “EntryD” é tratado como **Microsoft Entra ID**. “K4”, no contexto desta POC, é tratado como **CAV4/CA**. O Graph é a API do Entra ID; ele não é o mesmo serviço que o CAV4.

1. Visão simples do fluxo

A POC executa um login OIDC no CA/Entra e, depois do retorno do usuário, realiza duas fases independentes:

FASE 1 — CAV4 / User API e Admin API

- Usa o `access_token` obtido na troca do `authorization code`.
- Identifica o usuário por `userLogin`, normalmente uma matrícula/chave do CA.
- Consulta grupos, valores de informação, detalhes, `enterprise groups` e `roles`.

FASE 2 — Entra ID / Microsoft Graph

- Não reutiliza o token do CAV4.
- Obtém um segundo token, próprio da aplicação, pelo fluxo

`client_credentials` (app-only).

- Identifica o usuário pelo UPN/e-mail vindo nas `claims` do `id_token` do login.
- Consulta perfil, gerente, foto, cadeia de gestão, subordinados e grupos.

A falha de uma fase não interrompe a outra. Cada endpoint é executado individualmente; o resultado é armazenado como `ok=true/data` ou `ok=false/error`.

Arquivos principais:

- `backend/auth.py`: fluxo de login, `callback`, catálogo e orquestração.
- `backend/oidc.py`: `discovery`, autorização, **PKCE**, troca de tokens e validação.
- `backend/ca_client.py`: chamadas CAV4.
- `backend/graph_client.py`: token app-only e chamadas Graph.
- `backend/config.py`: variáveis de ambiente.
- `backend/session.py`: **state/nonce** temporários e resultado temporário.

3. Variáveis de ambiente

IMPORTANTE: nunca copie `client_secret` para código-fonte, documentação pública ou repositório. Use variável de ambiente/Secrets Manager.

2. Arquitetura e arquivos

Identidade da aplicação registrada no CA:

- **CA_CLIENT_ID**: client ID público da aplicação.
- **CA_CLIENT_SECRET**: segredo da aplicação confidential client.
- **CA_REDIRECT_URI**: callback exato registrado no CA, por exemplo:

<https://seu-dominio/auth/entra-callback>

- **CA_SCOPES**: escopos OIDC. Nesta POC: **openid profile**.

Discovery OIDC do CA:

- **OIDC_DISCOVERY_URL**: URL que termina em **/well-known/openid-configuration**.
- Alternativamente, informe **OIDC_ISSUER**, **OIDC_AUTHORIZATION_ENDPOINT**, **OIDC_TOKEN_ENDPOINT** e **OIDC_JWKS_URI** individualmente.
- **OIDC_VERIFY_SIGNATURE=true** deve ser mantido em ambientes reais.

API CAV4:

- **CA_API_BASE_URL**: host base, sem barra final. Exemplo conceitual:

<https://host-do-ca>

Microsoft Graph app-only:

- **GRAPH_TENANT_ID** ou **ENTRA_TENANT_ID**.
- **GRAPH_CLIENT_ID** ou **ENTRA_CLIENT_ID** ou **CA_CLIENT_ID**.
- **GRAPH_CLIENT_SECRET** ou **ENTRA_CLIENT_SECRET** ou **CA_CLIENT_SECRET**.
- **GRAPH_API_BASE_URL**: padrão **<https://graph.microsoft.com/v1.0>**.
- **GRAPH_AUTHORITY**: padrão **<https://login.microsoftonline.com>**.
- **GRAPH_SCOPE**: **<https://graph.microsoft.com/default>**.

A ordem de resolução das credenciais Graph é **GRAPH_***; depois **ENTRA_***; por fim **CA_***. Reutilizar **CA_*** só é correto se a mesma app tiver permissões de aplicação no Graph e consentimento administrativo.

4. Endereços e callbacks

Os endereços abaixo são os valores efetivamente configurados no arquivo `backend/.env.example` para o ambiente DSV. Eles não devem ser trocados por endereços genéricos sem validar a conectividade e o cadastro da aplicação.

2A.1 CAV4 / CA Petrobras

Servidor OIDC do CA (autenticação, discovery, authorization e token):

- RIC/rede interna/DSV: **<https://caauthz.petrobras.com.br>**
- Discovery usado nesta POC:

<https://caauthz.petrobras.com.br/well-known/openid-configuration>

- DMZ/rede externa, quando aplicável: **<https://authz.petrobras.com.br>**

Servidor REST do CAV4 (User API e Admin API):

- RIC/rede interna/DSV usado nesta POC: <https://fwca.petrobras.com.br>
- DMZ/rede externa, quando aplicável: <https://fwca-apps.petrobras.com.br>

As rotas CAV4 são anexadas ao **CA_API_BASE_URL**. Portanto, nesta POC, por exemplo, a chamada final é:

<https://fwca.petrobras.com.br/api/users/{userLogin}/user-groups>

2A.2 Entra ID / Microsoft Graph (EntryD/Entra ID)

A POC usa os endpoints públicos padrão do Microsoft Entra ID:

- Authority para emissão do token app-only:

<https://login.microsoftonline.com>

- Token endpoint final:

https://login.microsoftonline.com/{GRAPH_TENANT_ID}/oauth2/v2.0/token

- API Graph:

<https://graph.microsoft.com/v1.0>

- Scope usado no **client_credentials**:

<https://graph.microsoft.com/.default>

O **{GRAPH_TENANT_ID}** é o Directory (tenant) ID da organização. Não é o client ID e não deve ser substituído pelo nome do aplicativo.

Referências oficiais

Os endpoints privados do CAV4 dependem do ambiente corporativo e da documentação interna do CA. Por isso, os nomes formais dos scopes/roles do CAV4 não devem ser inventados: confirme-os com o time responsável pelo CAV4.

2A.3 Callback registrado no CAV4

Fluxo completo com frontend Next.js (recomendado para esta POC):

- URL exata a cadastrar no CA: <http://localhost:3000/auth/entra-callback>
- Variável: **CA_REDIRECT_URI**=<http://localhost:3000/auth/entra-callback>
- Quem recebe inicialmente: o proxy/rewrite do frontend Next.js.
- Destino real em desenvolvimento: backend FastAPI em

<http://127.0.0.1:8000/auth/entra-callback>.

O Next.js encaminha `/auth/*` para o backend sem o prefixo `/viewer`. Assim, o CA continua vendo a URL pública <http://localhost:3000/auth/entra-callback>, mas o processamento ocorre no FastAPI na porta 8000.

Fluxo somente backend, sem a tela `/viewer`:

- URL alternativa: <http://localhost:8000/auth/entra-callback>
- Nesse modo, defina **CA_REDIRECT_URI** com essa URL e cadastre-a também no CA.
- Inicie o login em <http://localhost:8000/auth/login>.

- Esse modo é útil para ver o JSON e os logs no terminal, mas não usa o frontend Next.js.

Callback pós-processamento para a tela:

- **VIEWER_PATH** padrão no backend: /viewer
- Redirecionamento final: /viewer?r={TOKEN}
- URL completa no fluxo local: **http://localhost:3000/viewer?r={TOKEN}**
- O TOKEN é temporário; não é um access token do CA nem do Graph.

Importante: /auth/entra-callback é callback de BACKEND. O frontend não possui um callback OIDC próprio. A página frontend é /viewer; ela apenas inicia o login por /auth/login e lê o resultado em /auth/result/{TOKEN}.

2A.4 Variáveis de ambiente do frontend

O frontend não precisa de client ID, client secret, tenant ou token. Em ambiente local, ele usa:

- **BACKEND_ORIGIN**: opcional; padrão **http://127.0.0.1:8000**.
- basePath fixo: /viewer (definido em frontend/next.config.ts).

O frontend acessa a mesma origem em **http://localhost:3000** e o Next.js faz o proxy das rotas /auth/* e /health. Nunca coloque **CA_CLIENT_SECRET**, **ENTRA_CLIENT_SECRET** ou qualquer token em NEXT_PUBLIC_* ou no código React.

2A.5 Variáveis de ambiente do backend

Obrigatórias para CAV4/OIDC:

- **CA_CLIENT_ID**
- **CA_CLIENT_SECRET**
- **CA_REDIRECT_URI**
- **OIDC_DISCOVERY_URL** (recomendado) OU

OIDC_AUTHORIZATION_ENDPOINT + **OIDC_TOKEN_ENDPOINT**

- **CA_API_BASE_URL**

Recomendadas/segurança:

- **CA_SCOPES=openid profile**
- **OIDC_VERIFY_SIGNATURE=true**
- **CA_SSL_USE_TRUSTSTORE=true**
- **CA_SSL_VERIFY=true**
- **CA_SSL_CERT_FILE** somente se for necessário informar um bundle PEM interno

Obrigatórias para consultas independentes do Graph:

- **ENTRA_TENANT_ID** (ou **GRAPH_TENANT_ID**)
- **ENTRA_CLIENT_ID** (ou **GRAPH_CLIENT_ID**)

- **ENTRA_CLIENT_SECRET** (ou **GRAPH_CLIENT_SECRET**)

Padrões que normalmente podem permanecer inalterados:

- **GRAPH_API_BASE_URL**=<https://graph.microsoft.com/v1.0>
- **GRAPH_AUTHORITY**=<https://login.microsoftonline.com>
- **GRAPH_SCOPE**=<https://graph.microsoft.com/.default>

O backend também aceita **CORS_ALLOW_ORIGINS** e **VIEWER_PATH**. Elas são opcionais.

CORS_ALLOW_ORIGINS só é necessário quando um navegador externo consumir a API.

TLS opcional:

- **CA_SSL_USE_TRUSTSTORE**=true.
- **CA_SSL_CERT_FILE**: bundle PEM da CA corporativa, se necessário.
- **CA_SSL_VERIFY**=true. Não desabilitar em homologação/produção.

5. Passo a passo completo do login

Nota de nomenclatura: neste documento, “Entra ID” é o nome oficial do serviço que às vezes é chamado informalmente de “EntryD”. O CAV4/CA Petrobras é o servidor de autenticação que inicia o login e entrega as claims; o Microsoft Graph é consultado separadamente pelo backend usando as credenciais da app do Entra ID.

URL completa do início do login nesta POC:

- <http://localhost:3000/auth/login> no fluxo com frontend;
- <http://localhost:8000/auth/login> no fluxo somente backend.

URL que o navegador visita no final do login:

- <http://localhost:3000/viewer?r={TOKEN}> no fluxo com frontend.

O parâmetro **r** não é enviado ao CA nem ao Graph. Ele referencia o resultado temporário armazenado pelo backend e é removido da barra do navegador depois que a página carrega o resultado.

3.1 GET /auth/login — início

A aplicação:

1. Valida se **CA_CLIENT_ID**, **CA_CLIENT_SECRET**, **CA_REDIRECT_URI** e discovery ou endpoints manuais estão configurados.
2. Gera **state** aleatório para proteção contra CSRF.
3. Gera **nonce** aleatório para validar o **id_token**.
4. Gera **code_verifier** e **code_challenge** usando **PKCE** S256.
5. Guarda **state**, **nonce** e **code_verifier** no armazenamento temporário.
6. Busca o discovery OIDC, se configurado.

7. Redireciona o navegador para `authorization_endpoint`.

Parâmetros enviados ao `authorization_endpoint`:

- `response_type=code`
- `client_id={CA_CLIENT_ID}`
- `redirect_uri={CA_REDIRECT_URI}`
- `scope={CA_SCOPES}`, normalmente **openid profile**
- `state`={valor aleatório}
- `nonce`={valor aleatório}
- `code_challenge`={SHA-256 base64url do `code_verifier`}
- `code_challenge_method=S256`

A URL de autorização é montada com QueryParams, portanto os valores são codificados corretamente.

3.2 GET /auth/entra-callback — retorno do provedor

Parâmetros recebidos na query string:

- `code`: authorization code de uso único e curta validade.
- **state**: deve ser igual ao **state** salvo no início.
- `error`: presente quando o provedor recusou o login.
- `error_description`: descrição opcional do erro.

Validações realizadas:

- Se `error` existir, o fluxo termina com erro categorizado.
- `code` e **state** são obrigatórios.
- **state** precisa existir no armazenamento temporário; depois é removido.
- O `code` é trocado por tokens.
- O `id_token` é validado por assinatura/JWKS, issuer, audience e **nonce**.

3.3 POST token_endpoint — troca do authorization code

Content-Type: `application/x-www-form-urlencoded`.

Parâmetros enviados:

- `grant_type=authorization_code`
- `code`={code recebido no callback}
- `redirect_uri`={mesmo **CA_REDIRECT_URI** usado na autorização}
- `client_id`={**CA_CLIENT_ID**}
- `code_verifier`={verifier salvo no início}
- `client_secret`={**CA_CLIENT_SECRET**}, pois a aplicação é confidential client

Resposta esperada do provedor:

- `id_token`: JWT com identidade e claims do usuário.
- `access_token`: token usado pela User/Admin API do CAV4 nesta POC.

- token_type, expires_in, scope e outros campos podem ser retornados conforme o provedor.

Nunca envie tokens ao frontend. Nesta POC eles ficam no backend.

3.4 Validação do id_token

Com **OIDC_VERIFY_SIGNATURE=true**:

- Baixa as chaves em jwks_uri.
- Valida assinatura com RS256 ou ES256.
- Valida audience contra **CA_CLIENT_ID**.
- Valida issuer contra o discovery.
- Valida **nonce** contra o **nonce** salvo.
- Rejeita token expirado, issuer/audience inválidos ou assinatura inválida.

As claims usadas pela POC:

- user_login, login, samaccountname ou onpremisesamaccountname: tentativas para encontrar a matrícula/chave usada pelo CAv4.
- preferred_username, upn, email ou unique_name: tentativas para encontrar o UPN/e-mail completo usado pelo Graph.
- name: nome exibido.
- email ou upn: e-mail exibido.
- claims: todas as claims são preservadas no resultado e impressas no log.

Atenção: userLogin e UPN são identificadores diferentes. userLogin normalmente é matrícula/chave do CAv4; UPN normalmente é e-mail completo.

6. Endpoints do CAv4

Todas as chamadas abaixo são GET, sem body, e enviam:

Authorization: Bearer {access_token_da_troca_OIDC}

Accept: application/json

O userLogin é inserido no path com URL encoding (quote, safe="").

4.1 Grupos do usuário

- Método: GET
- Caminho: /api/users/{userLogin}/user-groups
- Exemplo estrutural: GET {**CA_API_BASE_URL**}/api/users/USUARIO/user-groups
- Método Python: CAUserClient.user_groups(user_login)
- Finalidade: lista de grupos aos quais o usuário está associado.
- Resposta: JSON definido pela User API do CAv4; a POC não transforma o conteúdo, apenas o guarda em data.

4.2 Valores de informação

- Método: GET
- Caminho: /api/users/{userLogin}/information-values
- Método Python: CAUserClient.information_values(user_login)
- Finalidade: retorna valores de informação autorizados ao usuário.
- Resposta: JSON da User API, preservado sem alteração.

4.3 Detalhes administrativos do usuário

- Método: GET
- Caminho: /api/admin/users/{userLogin}
- Método Python: CAUserClient.admin_user_details(user_login)
- Finalidade: dados cadastrais/administrativos, podendo incluir lotação,

empresa, gerente/supervisor e outros atributos conforme o CA.

- É endpoint da Admin API e exige autorização administrativa correspondente.

4.4 Enterprise groups

- Método: GET
- Caminho: /api/admin/users/{userLogin}/enterprise-groups
- Método Python: CAUserClient.admin_enterprise_groups(user_login)
- Finalidade: lista de grupos corporativos/enterprise groups do usuário.
- É endpoint da Admin API.

4.5 Roles/papéis

- Método: GET
- Caminho: /api/admin/users/{userLogin}/roles
- Método Python: CAUserClient.admin_roles(user_login)
- Finalidade: lista de papéis/perfis atribuídos ao usuário.
- Não possui body.
- É endpoint da Admin API.

Resposta normalizada pela POC para cada consulta CAV4:

```
{
  "endpoint": "GET /api/...",
  "titulo": "...",
  "descricao": "...",
  "ok": true,
  "data": <JSON devolvido pelo CAV4>
}
```

Em caso de erro, data é substituído por error, contendo category, code,

message, cause, resolution e, quando aplicável, detail.

6.1 Permissões e autorização do CAV4

O código confirma que o `access_token` do login é usado para acessar:

- User API: user-groups e information-values.
- Admin API: detalhes, enterprise-groups e roles.

A implementação não contém nomes formais de scopes/roles do CAV4. Portanto, não é seguro inventar nomes como `user.read` ou `admin.read`. As permissões exatas precisam ser confirmadas no registro da aplicação e na documentação do CA.

Permissões funcionais necessárias:

1. Permissão de autenticação OIDC para login, com retorno de authorization code.
2. Permissão para a User API consultar grupos do usuário.
3. Permissão para a User API consultar information-values.
4. Permissão administrativa para consultar detalhes do usuário.
5. Permissão administrativa para consultar enterprise groups.
6. Permissão administrativa para consultar roles.

O fato de o token existir não garante autorização para todos os endpoints. O

CA pode responder 401 quando o token é inválido/expirado e 403 quando o usuário ou aplicação não têm autorização para o recurso.

Para migrar para outro projeto, confirme com o time do CAV4:

- scopes ou roles formais exigidos;
- se a autorização é de usuário, aplicação ou ambas;
- quais endpoints Admin podem ser chamados pelo client;
- host RIC/DMZ de cada ambiente;
- redirect URI cadastrada;
- discovery URL e realm, se houver.

7. Endpoints do Entra ID e Microsoft Graph

A POC obtém um token separado pelo fluxo **client_credentials**.

6.1 POST **{GRAPH_AUTHORITY}/{tenant}/oauth2/v2.0/token**

Exemplo estrutural:

POST **https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token**

Content-Type: application/x-www-form-urlencoded.

Parâmetros enviados:

- grant_type=**client_credentials**

- client_id={**GRAPH_CLIENT_ID**}
- client_secret={**GRAPH_CLIENT_SECRET**}
- scope=**https://graph.microsoft.com/.default**

Resposta consumida:

- access_token: token app-only do Microsoft Graph.

Esse token não representa um usuário. Por isso a POC não usa /me; usa /users/{upn}.

O token é memoizado dentro do GraphClient durante a execução do callback.

Todas as consultas Graph enviam:

Authorization: Bearer {token_app_only}

Accept: application/json

O UPN é codificado preservando @ e . (safe="@.").

6.2 Perfil do usuário

- GET /v1.0/users/{upn}?\$select={campos}
- Método Python: GraphClient.user()
- Finalidade: perfil completo do usuário.
- Campos solicitados:

id, displayName, givenName, surname, userPrincipalName, mail, jobTitle, department, companyName, officeLocation, mobilePhone, businessPhones, employeeId, employeeType, employeeOrgData, userType, preferredLanguage, usageLocation, accountEnabled, streetAddress, city, **state**, country, postalCode, faxNumber, ageGroup, createdDateTime, employeeHireDate, lastPasswordChangeDateTime, mailNickname, passwordPolicies, assignedLicenses, assignedPlans, onPremisesSamAccountName, onPremisesDistinguishedName, otherMails, proxyAddresses, imAddresses.

6.3 Gerente direto

- GET /v1.0/users/{upn}/manager?\$select={campos}
- Método Python: GraphClient.user_manager()
- Finalidade: gerente/supervisor direto.
- Campos solicitados: identificação, UPN, e-mail, cargo, departamento, empresa, localização, telefones, employeeId, employeeType, employeeOrgData, userType, city, **state**, country, accountEnabled e onPremisesSamAccountName.

- HTTP 404 é tratado como situação normal quando não há gerente definido.

6.4 Foto

- GET /v1.0/users/{upn}/photo/\$value

- Método Python: `GraphClient.user_photo()`
- Resposta original: binário de imagem.
- A POC converte para:

`contentType`: Content-Type recebido, padrão `image/jpeg`;

`sizeBytes`: tamanho em bytes;

`dataUri`: `data:{contentType};base64,{imagem}`.

- HTTP 404 ou corpo vazio retorna `null`.

6.5 Cadeia de gestão

- GET `/v1.0/users/{upn}/manager?$expand=manager($select={campos})&$select={campos}`
- Método Python: `GraphClient.user_management_chain()`
- Finalidade: gerente direto com o gerente dele aninhado em `manager`.
- Campos enxutos por pessoa: `id`, `displayName`, `userPrincipalName`, `mail`,

`jobTitle`, `department`.

6.6 Subordinados diretos

- GET `/v1.0/users/{upn}/directReports?$select={campos}`
- Método Python: `GraphClient.user_direct_reports()`
- Campos por item: `id`, `displayName`, `userPrincipalName`, `mail`, `jobTitle`,

`department`.

- Resposta usual do Graph é um objeto com `array value`.

6.7 Grupos/equipes do usuário

- GET `/v1.0/users/{upn}/memberOf?$select={campos}`
- Método Python: `GraphClient.user_member_of()`
- Campos solicitados: `id`, `displayName`, `description`, `mail`, `groupTypes`,

`securityEnabled`.

- Resposta usual do Graph é um objeto com `array value`.

Resposta normalizada pela POC para cada consulta Graph segue o mesmo formato de CAv4: `endpoint`, `titulo`, `descricao`, `ok` e `data` ou `error`.

8. Permissões do Entra ID e Microsoft Graph

A app registration usada pelo token **client_credentials** precisa de permissões de APLICAÇÃO (não Delegated), com admin consent concedido:

- **User.Read.All**

Usada para perfil `/users/{upn}`, gerente, cadeia de gestão, `directReports` e foto.

- **GroupMember.Read.All**

Usada para `/users/{upn}/memberOf` e leitura de grupos/equipes.

No portal do Entra:

1. App registrations > selecione a aplicação.
2. API permissions > Add a permission.
3. Microsoft Graph > Application permissions.
4. Adicione **User.Read.All** e **GroupMember.Read.All**.
5. Clique em Grant admin consent.
6. Crie um client secret e guarde somente o valor em Secret Manager/env.

HTTP 403 normalmente significa ausência de uma dessas permissões, tipo errado

(Delegated em vez de Application) ou falta de admin consent.

Não fazem parte desta POC:

- AuditLog.Read.All para signInActivity/último login.
- Permissões adicionais para outras APIs além das consultas descritas.

8.1 Orquestração e formato final

Depois de validar o `id_token`:

1. Extraí `userLogin` das claims para CAV4.
2. Extraí UPN/e-mail das claims para Graph.
3. Executa as cinco consultas CAV4 com o `access_token` da troca OIDC.
4. Executa as seis consultas Graph com o token **client_credentials**.
5. Monta:

```
{
  "status": "ok",
  "entra": {
    "userLogin": "...",
    "name": "...",
    "email": "...",
    "claims": { ... }
  },
  "ca": {
    "userLogin": "...",
    "userPrincipalName": "...",
    "user_groups": { ... },
    "information_values": { ... },
```

```

"admin_user_details": { ... },
"admin_enterprise_groups": { ... },
"admin_roles": { ... },
"graph_me": { ... },
"graph_manager": { ... },
"graph_photo": { ... },
"graph_management_chain": { ... },
"graph_direct_reports": { ... },
"graph_member_of": { ... }
}
}

```

Na prática, o campo `ca` contém tanto as cinco respostas CAV4 quanto as seis respostas Graph; o campo `fonte` do catálogo diferencia cada uma.

O backend imprime claims e resultados no log. Em produção, avalie mascarar PII, tokens, e-mails, telefones e dados organizacionais antes de gravar logs.

9. Front-end e rotas da própria POC

- GET `/auth/login`: inicia o login e redireciona ao `authorization_endpoint`.
- GET `/auth/entra-callback`: recebe `code/state`, troca tokens, valida `id_token`,

executa 11 consultas e redireciona para o visualizador.

- GET `/auth/result/{token}`: retorna o resultado temporário do callback.
- GET `/health`: health check.

O resultado e o `state` ficam em memória e expiram em aproximadamente 10 minutos.

Isso é adequado para POC, mas não para múltiplas instâncias em produção. Para produção, usar armazenamento compartilhado para `state/result` e uma sessão segura com cookies `HttpOnly/Secure/SameSite` apropriado.

11. Checklist para outro projeto

- [] Registrar a aplicação no CA e obter `CA_CLIENT_ID/SECRET`.
- [] Cadastrar `CA_REDIRECT_URI` exatamente igual à usada pelo novo sistema.
- [] Confirmar `OIDC_DISCOVERY_URL`, `realm/issuer`, `token endpoint` e `JWKS URI`.
- [] Confirmar scopes OIDC, incluindo `openid` e `profile`.
- [] Confirmar com o time do CAV4 os scopes/roles formais para User API.
- [] Confirmar autorização formal para cada endpoint Admin.
- [] Implementar Authorization Code + `PKCE` S256, `state` e `nonce`.

- [] Extrair userLogin/matricula das claims para o CAV4.
- [] Extrair UPN/e-mail completo das claims para o Graph.
- [] Registrar app Entra para acesso app-only ao Graph.
- [] Adicionar **User.Read.All** como Application permission.
- [] Adicionar **GroupMember.Read.All** como Application permission.
- [] Conceder admin consent.
- [] Guardar tenant, client ID e secret em variáveis seguras.
- [] Obter Graph token com **client_credentials** e scope .default.
- [] Enviar Bearer correto para cada API; não misturar os tokens.
- [] Codificar userLogin e UPN no path.
- [] Tratar 401, 403, 404 e expiração individualmente.
- [] Não expor access_token, client_secret ou id_token ao navegador.
- [] Revisar logs para evitar vazamento de PII e dados sensíveis.
- [] Testar em DSV antes de homologação/produção.

FIM DO DOCUMENTO